

Module 1: NumPy (Numerical Python)

NumPy provides high-performance multidimensional arrays (`ndarray`) and vectorized mathematical operations. It forms the computational backend for Scikit-Learn, PyTorch, and TensorFlow.

1.1 Array Creation & Inspection

```
import numpy as np

# 1. Array Initialization
a = np.array([1, 2, 3], dtype=np.float32)      # 1D Array
X = np.zeros((100, 4))                        # 100 samples, 4 features (initialized to 0)
y = np.ones((100, 1))                        # Target array initialized to 1
I = np.eye(3)                                # 3x3 Identity matrix (useful in linear regression)

# 2. Sequence Generators
grid = np.arange(0, 10, step=0.5)             # Evenly spaced by step size
linspace_grid = np.linspace(0, 1, num=50)      # Evenly spaced by number of samples

# 3. Array Inspection
print("Shape:", X.shape)                      # Output: (100, 4)
print("Dimensions:", X.ndim)                  # Output: 2
print("Data Type:", X.dtype)                  # Output: float64
```

1.2 Shape Manipulation & Reshaping

Machine learning algorithms strictly enforce input tensor dimensions. Reshaping without copying memory is critical.

```
arr = np.arange(12)                           # Shape: (12,)

# Reshape to 2D matrix (3 rows, 4 columns)
```

```

matrix = arr.reshape(3, 4)

# Dynamic Reshaping (-1 infers the missing dimension automatically)
feature_vector = arr.reshape(-1, 1)          # Shape: (12, 1) - Column vector
flattened = matrix.reshape(-1)              # Shape: (12,) - Flattened 1D array

# Transposition (Swapping axes)
X_transposed = matrix.T                    # Shape: (4, 3)

```

Check Step Success: Run `assert feature_vector.shape == (12, 1)` to confirm dimensions match target shape.

1.3 Indexing, Slicing & Boolean Masking

```

X = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])

# 1. Standard Slicing [rows, columns]
first_two_rows_all_cols = X[:2, :]          # Rows 0-1, All Columns
target_column = X[:, -1]                    # Extract last column (Shape: (3,))

# 2. Boolean Masking (Filtering outlier/specific values)
mask = X > 40
filtered_data = X[mask]                     # Returns 1D array of elements matching condition

# 3. Conditional Assignment (np.where)
# Usage: np.where(condition, value_if_true, value_if_false)
labels = np.where(X > 50, 1, 0)              # Binary classification thresholding

```

1.4 Vectorized Operations & Broadcasting

Broadcasting allows operations on arrays of different shapes without explicit looping.

```

# Rule: Dimensions are compatible when they are equal, or one of them is 1.

X = np.array([[1, 2, 3],
              [4, 5, 6]])          # Shape: (2, 3)
mean = np.array([1, 2, 3])        # Shape: (3,)

# Broadcasting mean subtraction across rows (Feature Centering)
X_centered = X - mean             # Output Shape: (2, 3)

# Element-wise operations
squared = X ** 2

exp_scores = np.exp(X)            # Used in Softmax activation

```

1.5 Linear Algebra for ML

```

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Element-wise multiplication
elem_mult = A * B

# Matrix Multiplication (Dot Product)
matrix_product = A @ B           # Equivalent to np.dot(A, B)

# Linear Regression Closed-Form Solution:  $\theta = (X^T X)^{-1} X^T y$ 
X = np.array([[1, 1], [1, 2], [1, 3]])
y = np.array([6, 5, 7]).reshape(-1, 1)

XTX_inv = np.linalg.inv(X.T @ X) # Inverse matrix
theta = XTX_inv @ X.T @ y        # Parameter vector

```

Check Step Success: Run `np.allclose(X.T @ X @ XTX_inv, np.eye(2))` to confirm the inverse matrix computation was accurate.

Module 2: Pandas (Data Manipulation & Feature Engineering)

Pandas provides high-level data structures (`Series` and `DataFrame`) for handling tabular data, missing values, and time-series conversions.

2.1 Data Loading & Exploratory Data Analysis (EDA)

```
import pandas as pd

# Load dataset
df = pd.read_csv("dataset.csv")

# Quick Inspection
print(df.head(5))           # First 5 rows
print(df.info())            # Column types, memory usage, non-null counts
print(df.describe())        # Summary statistics (mean, std, min, max, quartiles)
print(df.shape)             # (rows, columns)

# Unique values
print(df['category'].value_counts())  # Class distribution check
```

2.2 Selecting Data (`loc` vs `iloc`)

```
# loc: Label-based indexing [rows, columns]
subset_loc = df.loc[0:5, ['Age', 'Income']]  # Rows labeled 0-5 (inclusive), specific columns

# iloc: Integer-position based indexing [rows, columns]
subset_iloc = df.iloc[0:5, [0, 2]]           # Rows 0 to 4 (exclusive upper bound), columns at index 0 and 2

# Conditional Filtering
```

```
high_earners = df.loc[df['Income'] > 50000, ['Age', 'Education']]
```

2.3 Handling Missing Data & Feature Transformation

```
# Detect missing values
print(df.isnull().sum()) # Count missing values per column

# Imputation Strategies
df['Age'] = df['Age'].fillna(df['Age'].median()) # Median imputation for continuous variables

# Dropping missing targets
df = df.dropna(subset=['Target']) # Drop rows where target is missing

# Feature Transformation
# Mapping categorical strings to numerical integers
df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})

# One-Hot Encoding categorical features
df = pd.get_dummies(df, columns=['City'], drop_first=True)
```

2.4 Aggregations & GroupBy

```
# Mean metrics grouped by category
grouped = df.groupby('Target')['Income'].mean()

# Multi-metric Aggregations
agg_stats = df.groupby('Target').agg({
    'Age': ['mean', 'median'],
    'Income': ['min', 'max']
})
```

```
})
```

2.5 Converting Pandas DataFrame to NumPy Arrays for ML Models

```
# Extract features (X) and target (y)
X = df.drop(columns=['Target']).values      # Converts DataFrame to 2D NumPy array
y = df['Target'].values                    # Converts Series to 1D NumPy array

print("X shape:", X.shape)
print("y shape:", y.shape)
```

Check Step Success: Verify `isinstance(X, np.ndarray)` returns `True` before passing array inputs into Scikit-Learn estimators.

Module 3: Matplotlib (Visualization for Machine Learning)

Matplotlib allows diagnostic visualization of dataset distributions, model metrics, cost functions, and decision boundaries.

```
import matplotlib.pyplot as plt

# Recommended setup: Object-Oriented Interface
fig, ax = plt.subplots(figsize=(8, 5))
```

3.1 Essential ML Plots

1. Scatter Plot (Feature Relationships & Clusters)

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(df['Age'], df['Income'], c=df['Target'], cmap='coolwarm', alpha=0.7)
ax.set_title("Income vs Age by Target Class")
ax.set_xlabel("Age")
ax.set_ylabel("Income")
plt.tight_layout()
plt.show()
```

2. Line Plot (Training Loss / Cost Convergence Curves)

```
epochs = np.arange(1, 21)
loss = 1 / (epochs ** 0.5)

fig, ax = plt.subplots(figsize=(6, 4))
ax.plot(epochs, loss, label='Training Loss', color='red', linestyle='--', marker='o')
ax.set_title("Gradient Descent Convergence")
ax.set_xlabel("Epochs")
ax.set_ylabel("Loss (MSE)")
ax.grid(True)
ax.legend()
plt.show()
```

3. Histogram (Feature Distribution & Skewness Check)

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.hist(df['Income'], bins=20, color='skyblue', edgecolor='black')
ax.set_title("Feature Distribution: Income")
ax.set_xlabel("Income")
ax.set_ylabel("Frequency")
```

```
plt.show()
```

3.2 Multi-Panel Visualizations (Subplots)

```
# 1 Row, 2 Columns layout
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(12, 4))

# Plot 1: Age Histogram
axes[0].hist(df['Age'], bins=15, color='green', alpha=0.7)
axes[0].set_title("Age Distribution")

# Plot 2: Scatter Plot
axes[1].scatter(df['Age'], df['Income'], alpha=0.5)
axes[1].set_title("Age vs Income")

plt.tight_layout()          # Prevents title/axis overlapping
plt.show()
```

End-to-End Pipeline Workflow Example

This script demonstrates data generation, processing, target encoding, normalization, and visualization in a unified machine learning workflow.

```
import numpy as np, pandas as pd
import matplotlib.pyplot as plt

# 1. Generate Synthetic Dataset (NumPy)
np.random.seed(42)
num_samples = 100
```

```

data = {
    'Feature_1': np.random.normal(loc=10, scale=2, size=num_samples),
    'Feature_2': np.random.uniform(low=0, high=100, size=num_samples),
    'Category': np.random.choice(['GroupA', 'GroupB'], size=num_samples)
}

# 2. DataFrame Construction & Manipulation (Pandas)
df = pd.DataFrame(data)

# Handle potential missing values
df['Feature_1'] = df['Feature_1'].fillna(df['Feature_1'].mean())

# Categorical Encoding
df['Category_Code'] = df['Category'].map({'GroupA': 0, 'GroupB': 1})

# Construct Target Vector (Synthetic Binary Output)
df['Target'] = np.where(df['Feature_1'] + (df['Feature_2'] / 20) > 12, 1, 0)

# 3. Extract & Preprocess Arrays (NumPy Vectorization)
X = df[['Feature_1', 'Feature_2']].values
y = df['Target'].values

# Standard Scaling (Z-score normalization: (X - mu) / sigma)
X_scaled = (X - np.mean(X, axis=0)) / np.std(X, axis=0)

# 4. Model Diagnostic Visualization (Matplotlib)
fig, ax = plt.subplots(figsize=(7, 5))
scatter = ax.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y, cmap='bwr', alpha=0.8, edgecolors='k')
ax.set_title("Standardized Feature Space")
ax.set_xlabel("Feature 1 (Standardized)")
ax.set_ylabel("Feature 2 (Standardized)")
fig.colorbar(scatter, label='Class Label')
ax.grid(True)
plt.show()

```